PORTFOLIO PROJECT — STAGE 3 (FRANCE)

Technical Documentation

Project: CampOrga — a platform for managing coding & tech summer camps

School: Holberton School — France

Team: Thélyaan Dufrénoy & Collins Odi Obi

Status: draft to be validated by the team before requesting QA review

1. User Stories and Mockups

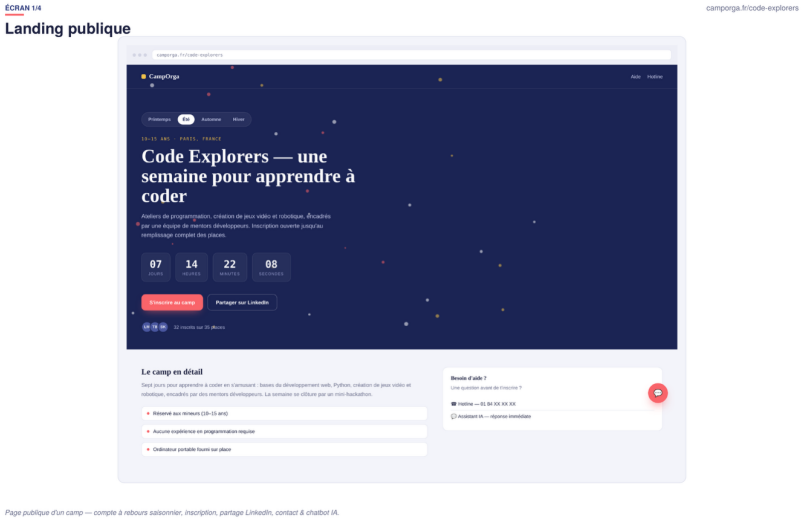
1.1 User stories (MoSCoW)

The stories below translate the use cases already validated with the client into concrete, testable statements, prioritized for the MVP.

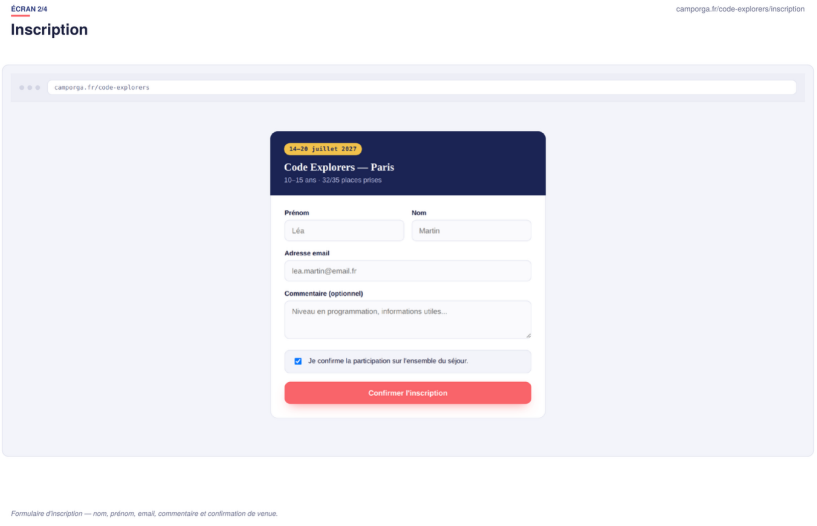
Priority	User story
Must Have	As an organizer, I want to create a camp with its dates, description and eligibility criteria, so that I can publish it to participants.
Must Have	As an organizer, I want a public landing page generated automatically for my camp, so that I don't have to build one manually.
Must Have	As a visitor, I want to register for a camp in a few simple steps, so that I can secure my spot quickly.
Must Have	As a super admin, I want to create and manage user accounts and roles, so that I control who can create and manage camps.
Must Have	As a super admin, I want to export a database backup in SQL or CSV, so that the organization's data stays safe.
Should Have	As an organizer, I want to see the list of participants registered to my camp, so that I can prepare logistics.
Should Have	As a visitor, I want to share a camp's landing page on LinkedIn, so that I can help spread the word.
Should Have	As a visitor, I want to contact the organization via a hotline or chatbot, so that I can get quick answers before registering.
Could Have	As a visitor, I want the landing page's visual theme to adapt to the current season, so that the experience feels fresh.
Could Have	As an organizer, I want to see recent registrations directly on my dashboard, so that I can follow up quickly.
Won't Have (MVP)	As a visitor, I want to pay for my registration online — deferred to a future iteration.
Won't Have (MVP)	As a participant, I want a native mobile app — deferred to a future iteration.

1.2 Mockups

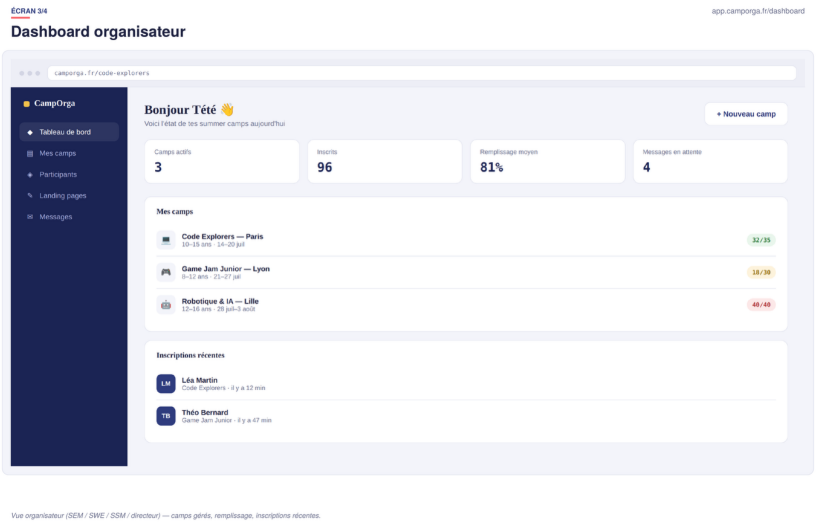
The four screens below were validated with the client as an interactive HTML prototype. Note: the visual identity will change before final delivery, since it must not reuse Holberton School's graphic charter (client feedback, see the Stage 1 report).



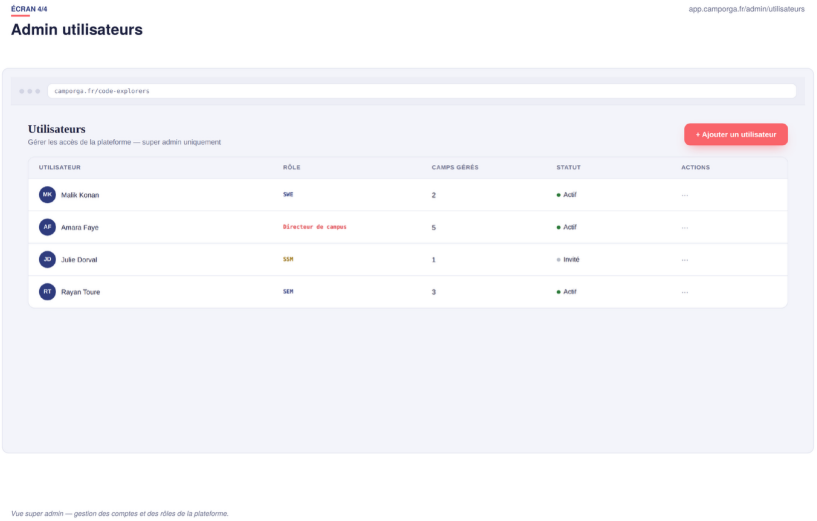
Public landing page — countdown, camp details, registration CTA.



Registration form — first name, last name, email, comment, confirmation.



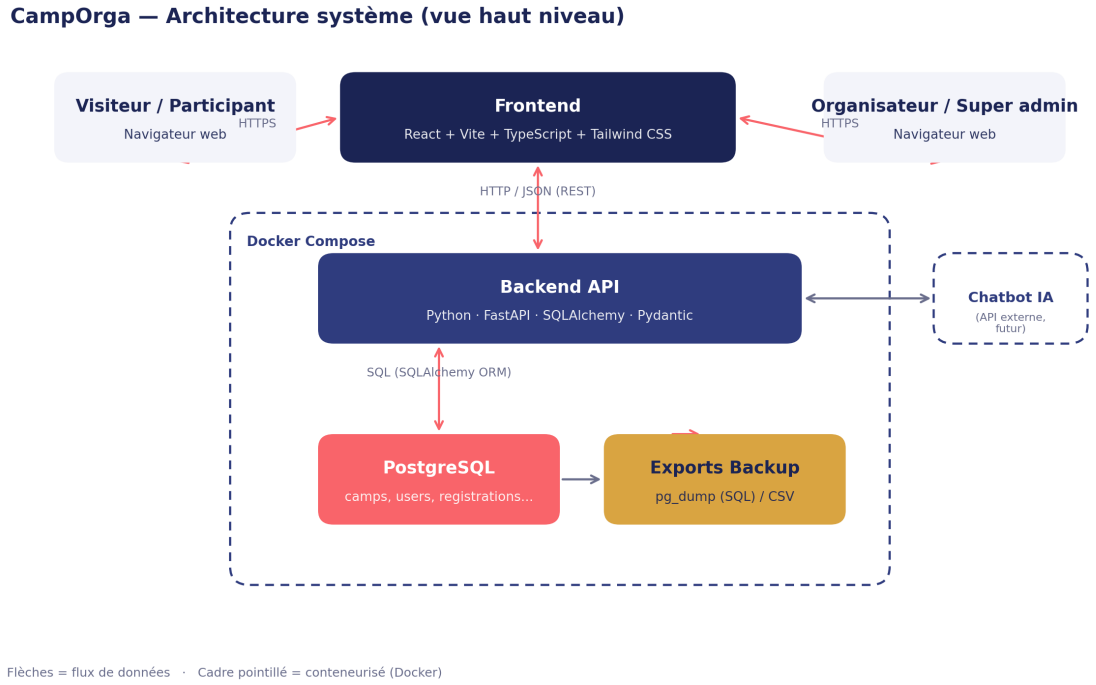
Organizer dashboard — managed camps and recent registrations.



Super admin — user and role management.

2. System Architecture

CampOrga follows a classic three-tier architecture: a React frontend, a FastAPI backend exposing a REST API, and a PostgreSQL database. The backend and database are containerized together with Docker Compose, so the whole stack starts with a single command in any environment (development or production).



High-level architecture — arrows show the direction of data flow.

- The frontend never talks to the database directly — all data passes through the backend's REST API, which validates and controls access (organizer-level roles — sem, swe, ssm, directeur_campus, directeur_technique — vs. super_admin vs. public).
- Backup exports (SQL via pg_dump, CSV via SQL queries) are generated by the backend and stored as files the super admin can download.
- An AI chatbot is shown as an optional external service: it is out of scope for the MVP but the architecture leaves room for it without any redesign.

3. Components, Classes, and Database Design

3.1 Backend — models and services

Class / Module	Responsibility
User (model)	Maps to the users table. Attributes: id, first_name, last_name, email, password_hash, role, status, created_at. role is one of super_admin, sem, swe, ssm, directeur_campus, directeur_technique (sem is the role currently used as camp organizer in testing); status is one of active, invited, disabled.
Camp (model)	Maps to the camps table — the central entity. Attributes: id, organizer_id, name, slug, description, start_date, end_date, city, age, capacity, status, created_at. status is one of draft, published, archived.
CampModality (model)	Eligibility criteria attached to a camp. type is one of age_restriction, education_level, other.
LandingPage (model)	Public page content for a camp (season theme, hero title/description, published_at). season_theme is one of printemps, ete, automne, hiver.
Registration (model)	A visitor's registration to a camp (first_name, last_name, email, comment, confirmed, created_at).
ContactMessage (model)	A message sent via the hotline or chatbot channel (name, email, message, channel, created_at), optionally linked to a camp (camp_id set to null if the camp is later deleted).
Backup (model)	History of backup exports (format, triggered_by, file_path, file_size_kb, created_at). format is one of sql, csv.
CampService	create_camp(), update_camp(), list_camps(), get_by_slug() — business logic for camp management.
RegistrationService	register_participant(), list_registrations() — validates capacity before accepting a registration.
UserService	create_user(), assign_role(), authenticate() — account and access management.
BackupService	export_sql(), export_csv() — wraps pg_dump and CSV generation, logs each export.

3.2 Frontend — main components

Component	Responsibility
LandingPage	Renders a camp's public page: hero, countdown, eligibility, registration CTA.
CountdownWidget	Computes and displays time remaining until the camp start date; visual style depends on the season.
RegistrationForm	Collects and submits registration data; shows validation errors and confirmation.
OrganizerDashboard	Lists the organizer's camps with fill rate, and recent registrations.
CampForm	Create/edit a camp (dates, description, modalities).
AdminUsersTable	Super admin view — list, add, and edit users and their roles.
AuthContext	Holds the current session (role, token) and guards protected routes.

3.3 Database design — ER diagram

PostgreSQL, 7 tables, camps as the central entity. Full SQL script: docs/database-schema.sql.

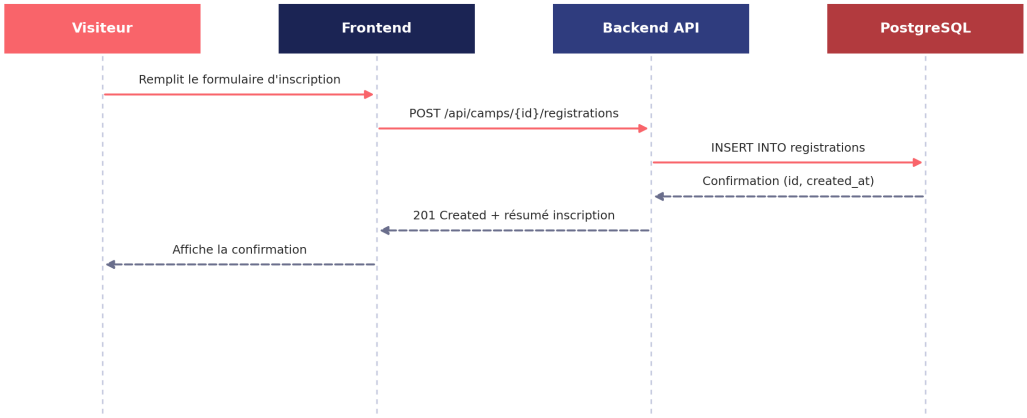


Entity-relationship diagram — PK in coral, FK in gold, UK in navy.

4. Sequence Diagrams

The three diagrams below illustrate the key interactions between the frontend, the backend API, and the database for the MVP's most critical flows.

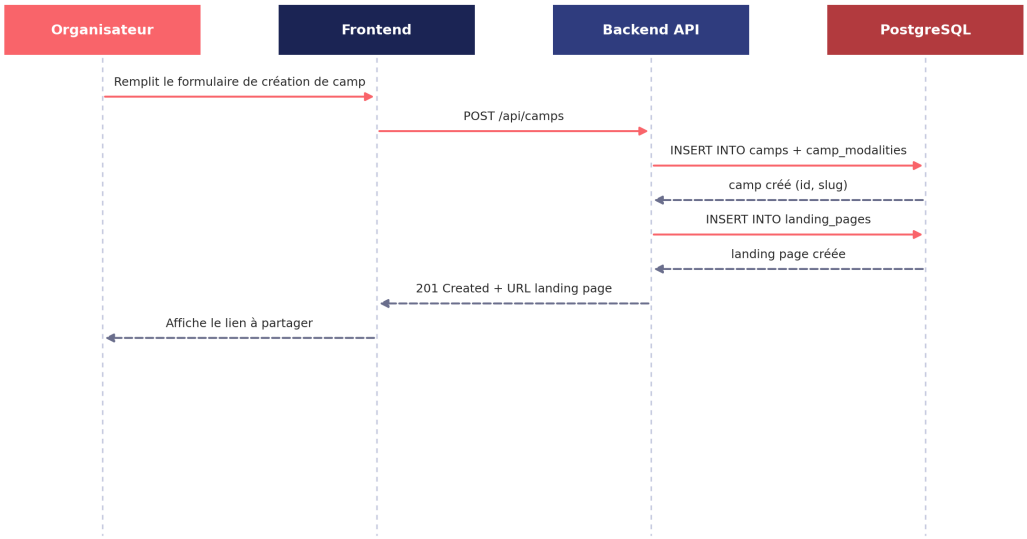
Séquence 1 — Un visiteur s'inscrit à un camp



Cas d'usage : UC11 — S'inscrire à un camp (voir liste des cas d'usage).

A visitor registers for a camp (UC11).

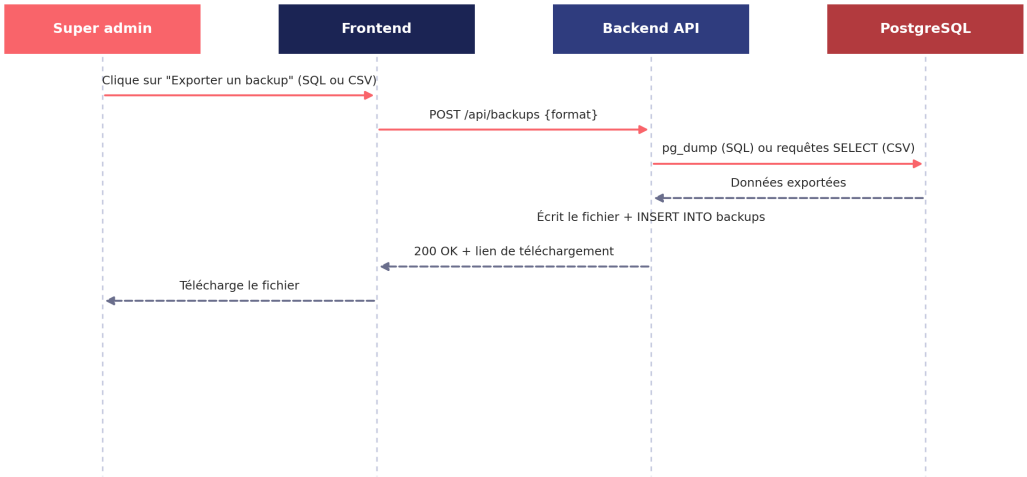
Séquence 2 — Un organisateur crée un camp et sa landing page



Cas d'usage : UC5 — Créer un summer camp, UC7 — Générer la landing page.

An organizer creates a camp and its landing page (UC5, UC7).

Séquence 3 — Le super admin exporte un backup



Cas d'usage : fonctionnalité de backup demandée par le client (table backups).

The super admin exports a database backup (SQL/CSV).

5. API Specifications

5.1 External services

Service	Purpose	Why this choice
Claude API or OpenAI API	Powers the landing page's AI chatbot (out of scope for the MVP, planned for a future iteration).	Mature REST APIs, easy to call from a single backend endpoint, no dedicated ML infrastructure needed.
LinkedIn share link	Lets a visitor share a camp's landing page.	A simple share URL (no authentication, no SDK) — not a true API integration, kept intentionally lightweight.

5.2 Internal API — key endpoints

Method	Path	Input	Output
POST	/api/camps	JSON: name, dates, description, modalities (organizer/admin only)	201 — created camp (id, slug)
GET	/api/camps	Query params: status, organizer_id (optional)	200 — list of camps
GET	/api/camps/{slug}/public	—	200 — public camp + landing page data
PATCH	/api/camps/{id}	JSON: fields to update	200 — updated camp
POST	/api/camps/{id}/registrations	JSON: first_name, last_name, email, comment, confirmed	201 — created registration
GET	/api/camps/{id}/registrations	— (organizer/admin only)	200 — list of registrations
POST	/api/auth/login	JSON: email, password	200 — access token
POST	/api/users	JSON: first_name, last_name, email, role (super admin only)	201 — created user
GET	/api/users	— (super admin only)	200 — list of users
POST	/api/backups	JSON: format (sql | csv) (super admin only)	200 — download link + backup record

All endpoints return standard HTTP status codes (400 for validation errors, 401/403 for auth issues, 404 when not found). Full interactive documentation is auto-generated by FastAPI at /docs (Swagger UI).

6. SCM and QA Plans

6.1 Source control management

- Branches: main (stable, deployable), develop (integration), and one feature/* branch per task (e.g. feature/camp-creation-api).
- Commit convention: Conventional Commits (feat:, fix:, docs:, chore:...) for a readable history.
- Every feature branch is merged into develop via a pull request; with a 2-person team, each PR is reviewed by the other member before merging.
- main is only updated from develop at each delivery milestone (end of Stage 4, Stage 5).

6.2 Quality assurance

- Backend: unit and integration tests with Pytest (models, services, API endpoints).
- Frontend: component tests with Vitest and React Testing Library.
- Manual API testing during development via the Swagger UI (/docs) and Postman for edge cases.
- Linting and formatting enforced with Ruff + Black (Python) and ESLint + Prettier (TypeScript) before each merge.
- GitHub Actions runs the test suite automatically on every push and pull request.
- Staging: local Docker Compose environment. Production: deployment to Railway or Render.

7. Technical Justifications

Decision	Justification
React + TypeScript (frontend)	Industry-standard for interactive UIs; TypeScript catches errors early. Aligns with the team's ongoing learning goals for this project.
FastAPI (backend)	The team already knows Python; FastAPI is lightweight, fast to learn, and auto-generates API documentation — valuable given the tight delivery deadline.
PostgreSQL	Relational data (camps, users, registrations) with clear relationships benefits from a relational database; the team already has SQL experience.
Docker / Docker Compose	Guarantees the same environment for both team members and for deployment, removing "works on my machine" issues — important with a reduced 2-person team.
SQL + CSV backup (not a managed backup service)	Directly requested by the client; a self-hosted export keeps the MVP simple and avoids an extra paid dependency.
No online payment / no native app (out of scope)	Keeps the MVP achievable within the first-week-of-September deadline; explicitly deferred rather than dropped.
Frontend shared between 2 roles instead of a dedicated Frontend developer	Adapted after Harold's departure from the team; mitigated by keeping the frontend scope intentionally simple (see mockups).